

**Universidad Torcuato Di Tella**

**Master MiM + Analytics**



100/100

## **TRABAJO FINAL**

### **Integrantes del grupo:**

- Sarkissian, Jonathan David
- Reina, Marcos
- Boca, Martino

**Materia:** Toma de Decisiones Secuenciales y Aprendizaje por Refuerzo

**Profesor:** Daniela Pucci de Farias

**AÑO: 2025**

## Propuesta

La optimización del portafolio constituye un pilar fundamental en la gestión financiera, enfocándose en la asignación eficiente de activos bajo incertidumbre. Dada la naturaleza secuencial y estocástica de las decisiones de inversión, los Modelos de Decisión de Markov (MDP) ofrecen un marco analítico idóneo. Estos permiten formular el problema como una búsqueda de políticas óptimas que guían la evolución del portafolio a través de estados y acciones, integrando las preferencias del inversor y la dinámica del mercado. A continuación, se presenta nuestro modelo MDP desarrollado para este fin.

Proponemos expandir el modelo de optimización del portafolio visto en clase incorporando más elementos al modelo. En particular se propone un modelo de programación dinámica de Optimización del Portafolio con Consumo y Aversión al riesgo incorporando, además:

- Tasa de descuento. A mayor horizonte menor es el valor de la riqueza actual.
- Costos de transacción. Invertir en el activo riesgoso tiene una comisión de transacción al invertir. Esto típicamente la incorporan los banco o corporaciones financieras al invertir. El activo libre de riesgo no tiene costo de transacción.

El modelo propuesto es una extensión más realista del original ya que incorporamos consumo, aversión al riesgo, tasas de descuento y costos de transacción. Esto permite captar mejor la dinámica real financiera. Sin embargo, al ser un modelo más complejo, computacionalmente se vuelve más difícil de resolver.

## Modelado

Tenemos el siguiente modelo de optimización del portafolio con consumo y aversión al riesgo:

Estado: valor del portafolio  $W_{t+}^r$  del activo riesgoso y  $W_{t+}^f$  del activo libre de riesgo. Elegimos estas dos variables de estado para poder modelar el flujo de retorno de ambos activos por separado. Esto es porque para el activo riesgoso hay costos de transacción que para poder calcularlos es necesario saber cuál es el valor del activo riesgoso en cada momento para un  $\beta$  y  $\delta$ . Esto aumenta la complejidad del problema y el espacio de estados, por lo que tratamos de discretizarlos lo menos posible para evitar que el espacio de búsqueda sea excesivamente grande y más costoso computacionalmente.

Acción:  $0 \leq \beta_t \leq 1$ ,  $0 \leq \delta_t \leq 1$ . En cada periodo  $t$ , consumimos  $\delta_t * W_t$  e invertimos una fracción  $\beta_t$  del restante en el activo riesgoso y  $1 - \beta_t$  en el activo libre de riesgo. De igual manera que para el espacio de estados, el espacio de acciones se duplica y por lo tanto aumenta la complejidad computacional. Para mitigar esto tratamos de no discretizar demasiado el espacio.

$r^r$  es el retorno esperado del activo riesgoso.

$r^f$  es el retorno esperado del activo libre de riesgo.

$C_r$  es el Costo of Service para el activo riesgoso que cobra el banco y se expresa en porcentaje sobre el total invertido. En general se cobra solo para el activo riesgoso (por ejemplo, acciones) mientras que el activo libre de riesgo no tiene costo. Esto lo calculamos en cada momento sobre el cambio en el valor de la composición de las acciones sea que se compren o vendan.

La tasa de descuento  $\alpha = e^{-r^f}$ . Esto es una tasa continua de descuento muy usada en finanzas.

$\epsilon_t \sim N(0,1)$  es el factor aleatorio del precio a lo largo del tiempo.

Recompensa:  $v(\delta W) = (\delta W)^\gamma$ . La función de recompensa es la fracción del valor total de todos los activos destinada al consumo. Cuanto mayor es la proporción del valor destinada al consumo mayor es la recompensa. Sin embargo, para modelar la aversión al riesgo aplicamos una función de utilidad del tipo  $f(x) = x^\gamma$  que permite modelar retornos marginal decreciente. Es decir, cuanto más consumo cada vez, esto aporta cada vez menos utilidad si  $\gamma < 1$ . Para nuestro caso dejamos fijo  $\gamma = 0.5$  para tener una función de aversión al riesgo moderada.

Ecuación de Bellman:

$$V^*(W) = \max_{\beta, \delta} \{v(\delta W) + \alpha E_{\epsilon \sim N(0,1)} [V^*(W_{t+}^r, W_{t+}^f)]\}$$

$$\text{con } v(\delta W) = (\delta W)^Y \text{ y con } W = (W_{t+}^r, W_{t+}^f) = W_{t+}^r + W_{t+}^f$$

y las transiciones:

$$W_{t+}^r = [\beta(1 - \delta)(W_t^r + W_t^f) - C_r] \beta(1 - \delta)(W_t^r + W_t^f) - W_t^r \Big] e^{r_r - \sigma_r^2/2 + \sigma_r \epsilon_t}$$

$$W_{t+}^f = (1 - \beta)(1 - \delta)(W_t^r + W_t^f) e^{r_f}$$

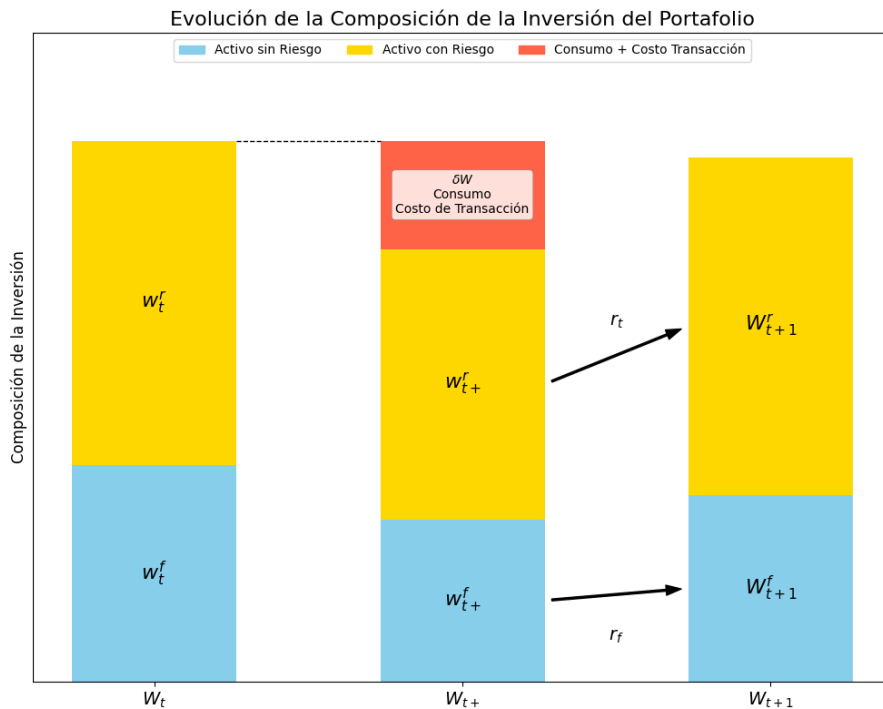
Criterio de optimalidad:

$$\max_u E [\alpha v(\delta_T W_T^Y)] = \max_u E [\alpha \delta_T W_T^Y].$$

$$\max_u \left[ \sum_{t=0}^T \alpha^t v(\delta_t W_t) \right]$$

Es decir, buscamos maximizar la rentabilidad total del portafolio a lo largo del tiempo descontando los flujos futuros, considerando los costos de transacción asociados. También una parte puede ser consumida o invertida por lo que debemos encontrar la decisión óptima entre inversión y consumo tal que maximizar la ganancia total. Por último, al modelar como una función de utilidad incorporamos aversión relativa constante al riesgo, es decir, tener más riqueza para consumo rinde cada vez menos por lo tanto menos riesgo quiere asumir el inversor.

El siguiente grafico ilustra la evolución de la composición de los activos:



*Ilustración 1: Gráfico composición del portafolio*

En el grafico 1 se observa que al principio en un momento  $t$  dado, se tiene un valor de  $W_t^r$  en el activo riesgoso,  $W_t^f$  en el activo libre de riesgo y un total de  $W_t$ . Luego se decide una nueva composición  $\beta$  y  $\delta$ . Al total de la composición  $W_{t+}$  se le resta una parte dedicada al consumo  $\delta$  y otra del costo de transacción  $C_r$  del activo riesgoso sea que se venda parte del activo o se compre una porción mayor. El neto restante se invierte en el activo riesgoso  $W_{t+}^r$  y el resto se invierte en el activo libre de riesgo con valor  $W_{t+}^f$ . Luego, se produce un resultado aleatorio con el precio del activo riesgoso  $r_t^r$  generado un nuevo valor de  $W_{t+1}^r$  y  $W_{t+1}^f$  para el libre de riesgo cuya tasa de crecimiento es constante  $r_t^f$ . En resumen, los estados  $W_t^r$  y  $W_t^f$  van evolucionando de acuerdo con los  $\beta$  y  $\delta$  seleccionados y las simulaciones de los retornos del activo riesgoso  $\epsilon_t$  lo cual a su vez lleva estos estados a nuevos estados llamados  $W_{t+1}^r$  y  $W_{t+1}^f$ . A partir de estos nuevos estados se extrae el valor de la función valor  $V^*(W_{t+1}^r, W_{t+1}^f)$  de estar en cada uno

Una consideración adicional para el modelado de este modelo es la función de crecimiento del espacio de estados y acciones. Ambos crecen de la siguiente forma:

$$|S| = N_s^{D_s} \text{ y } |A| = N_a^{D_a}$$

Con  $N_s$  siendo la cantidad de discretizaciones del espacio de estados y  $D_s$  la cantidad de dimensiones del estado (que en este caso es la cantidad de activos que son 2 en total). Y  $N_a$  siendo la cantidad de discretizaciones del espacio de acciones y  $D_a$  la cantidad de dimensiones de la acción (decisiones a tomar que en nuestro caso son consumo e inversión). Es decir, el crecimiento en el espacio de estados y acciones es exponencial en  $D_s$  y  $D_a$ . Esto quiere decir que agregar más activos (a parte del libre de riesgo y el riesgoso) y agregar más decisiones a tomar (por ejemplo, destinar un % del valor total a donar) aumenta exponencialmente el espacio de búsqueda lo cual provoca que la resolución del problema sea inviable para grandes valores de  $D_s$  y  $D_a$ . Para esto nos limitamos a solo dos activos y dos decisiones a tomar, es decir,  $D_s = 2$  y  $D_a = 2$ . Vale la pena destacar, que manteniendo fijos  $D_s$  y  $D_a$  el crecimiento del espacio de estado y acciones cuando discretizamos más es polinomial, pero aun así puede ser costoso para altos valores de  $D_s$  y  $D_a$ . 

## Implementación Computacional y Análisis

Para resolver este problema implementamos los siguientes 3 algoritmos:

- Iteración Valor
- Iteración Política
- Q – Learning (con simulación de transiciones)

Antes de aplicar los algoritmos, definimos los parámetros del modelo. Todas las tasas son mensuales:

- $R_f = 0.0034$  (fijo Bono del tesoro)
- $R_r = 0.035$  (fijo del S&P 500)
- $C_r = 0.01$  (fijo)
- $\gamma = 0.5$  (fijo)
- $\sigma_r = 0.033$  (fijo del S&P 500)
- $\alpha = e^{-0.0034} = 0.997$  (variable)

Todos los datos fueron extraídos de los valores actuales del mercado americano en tasas mensuales. Con estos valores arrancamos inicialmente todos los algoritmos. El parámetro  $\alpha$  lo cambiaremos para evaluar el rendimiento de los algoritmos y los resultados, los demás quedan constantes.

Al principio discretizamos  $W_t^f$  y  $W_t^r$  en un espacio entre 0 y 1.000.000 con valores de 10.000 en 10.000 (total 101 de discretizaciones). También discretizamos el  $\beta$  y  $\delta$  con valores entre 0 y 1 que van de 0.05 en 0.05 (total 21 discretización para cada una). Tratamos de discretizarlo lo menos posible para evitar agrandar excesivamente el espacio de estados. En este caso el espacio de estados es de 10.201 ( $101^2$ ) valores y el espacio de acciones es de 441 ( $21^2$ ). La combinaciones entre estado acción son de 4,5 Millones! Luego, nos dimos cuenta que esto era demasiado grande para el problema para poder interpretarlo (e implementarlo) y decidimos reducir las discretizaciones del espacio de estados a 10, es decir, de 100.000 en 100.000 lo cual es más manejable. En este caso el espacio total de estado y acciones es  $|S||A| = 11^2 * 21^2 = 53.361$  combinaciones. Buena solución.

## Análisis de Resultados

### Iteración de Valor

Para la implementación y el cálculo de la ecuación de Bellman, tuvimos que simular de antemano distintos valores de  $\epsilon \sim N(0,1)$  y luego computar la función valor de la siguiente manera

$$V^*(W) = \max_{\beta, \delta} \left\{ v(\delta W) + \alpha \frac{1}{N_{obs}} \sum_{\epsilon_i \in N_{obs}} [V^*(W_{t+}^r, W_{t+}^f)] \right\} \quad \checkmark$$

Es decir, computamos el valor promedio de la función valor para un  $\beta$  y  $\delta$  dados sobre todas las simulaciones de los  $\epsilon$  que son  $N_{obs}$ . Si simulamos demasiado podría ralentizar el cómputo de la función valor por lo que usamos simulaciones relativamente bajas.

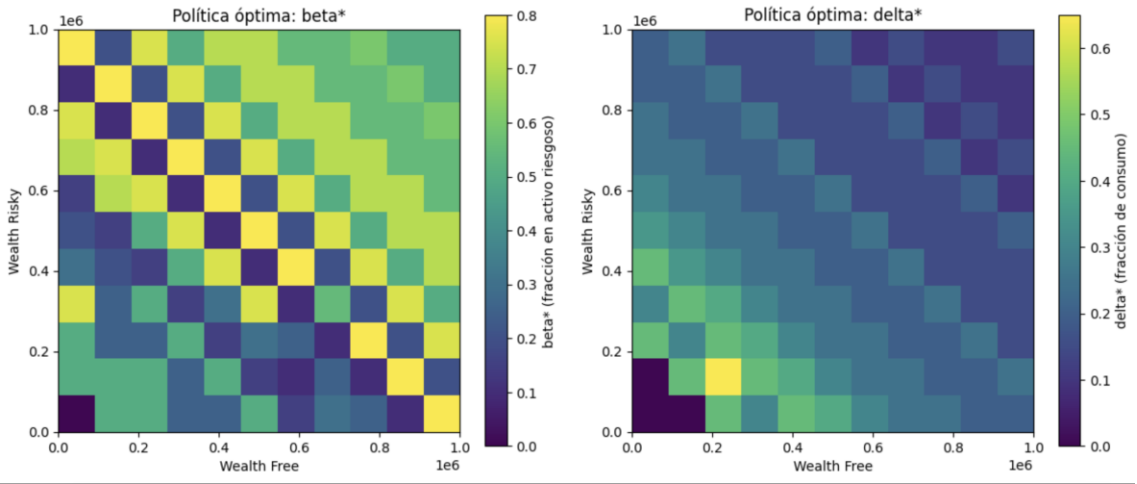
Iteración valor hará muchas pasadas evaluando en cada una cada  $W_t^f$  y  $W_t^r$  y computando para cada combinación de estados, todas las combinaciones de acciones  $\beta$  y  $\delta$  y seleccionar la maximiza la ecuación de Bellman. El cómputo en cada pasada es relativamente rápido, pero serán necesarias muchas pasadas hasta lograr convergencia sobre todo si el  $\alpha$  es alto.

Inicialmente cuando planteamos el modelo con una grilla de estas (Wf,Wr) con puntos muy cercanos y con la posibilidad de evaluar muchas distintas acciones es decir muchos beta y delta el modelo se volvió demasiado pesado y se volvía imposible de correr. Fue por esto por lo que buscamos simplificar el modelo para que pudiese correr en tiempos razonable. Comenzamos limitando la grilla de estados para disminuir la cantidad de veces que se computaba la función valor. Luego también tuvimos que reducir la cantidad de acciones disminuyendo ahí también el número de opciones. Además, utilizamos la librería numba que ayuda a ejecutar el cómputo más rápido al usar lenguaje de máquinas. Con estos cambios logramos empezar a poder correr el modelo en un par a de horas, hasta poder correrlo en unos 30 minutos. Claro está que todas estas simplificaciones en el modelo provocan peores resultados

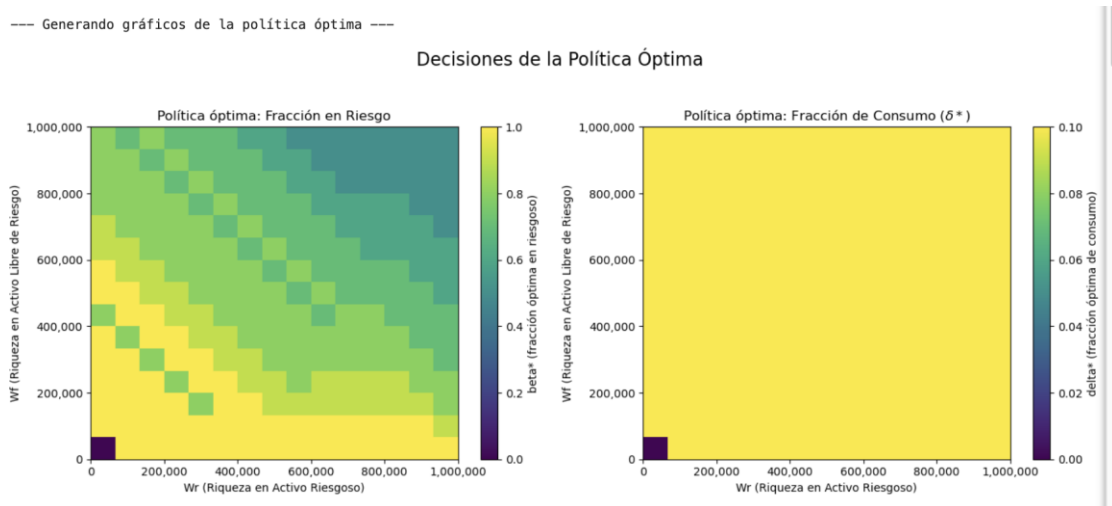
En un principio la política para la tasa de consumo delta se puede ver en el grafico que solía ser bastante baja para la gran mayoría de los estados, solo para cuando los estados en donde tanto la cantidad en riesgo como la libre de riesgo son de aproximadamente 0,2 se llega a valores más altos de delta de aproximadamente de entre 0,4-0,6

En cambio, para fracción del capital, la política óptima se ve que no siguen un patrón muy claro, sino que sobre la diagonal se mezclan valores altos y bajo de beta. En cambio, cuando los valores de Wf y Wr son ambos altos o bajos, se adoptan valores de beta cercanos a 0,5. Como estos resultados no fueron muy convincentes decidimos modificar algunos parámetros del modelo para mejorarlo.

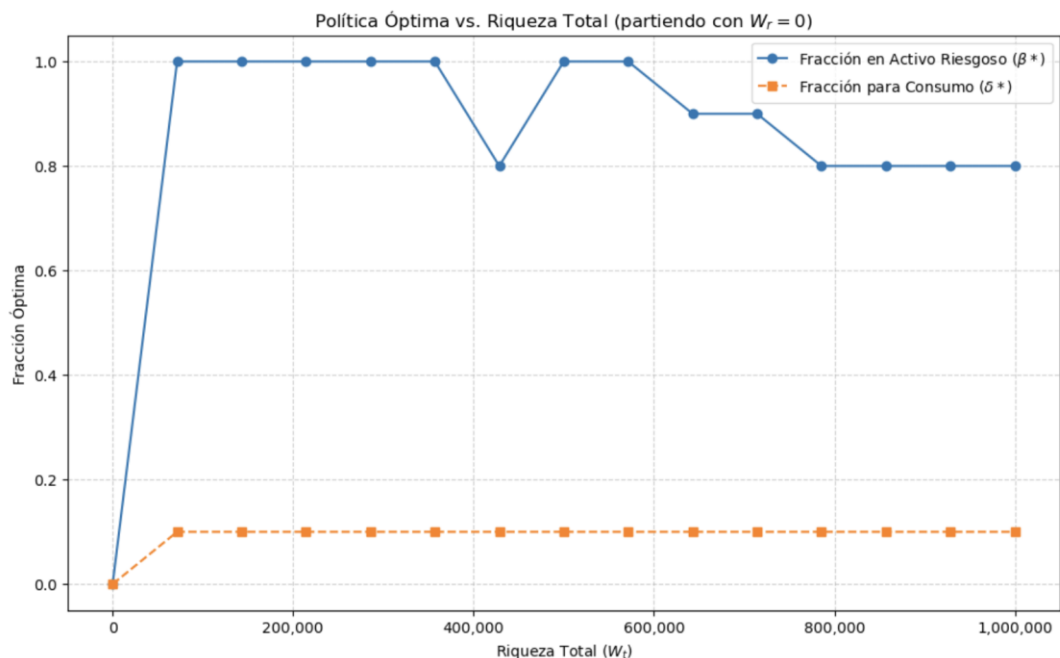
Por qué piensan que pasó esto?



Luego de algunas iteraciones modificando la cantidad de estados, de acciones, el delta y las tasas logramos llegar a una política más realista y estructurada. En la gráfica debajo se puede ver la política más ordenada de lo que aprendió el modelo. Para cuando tiene poco capital prefiere invertir fuerte en el activo más riesgoso, buscando maximizar sus ganancias. Pero a medida que el capital aumenta se vuelve más conservador y prefiere no arriesgar tanto su dinero. En cuanto al consumo siempre que tenga dinero para consumir decide consumir la misma proporción un 10%



En el siguiente grafico se ve de una manera más clara lo aprendido por el modelo. Cuando tiene poco capital busca tomar riesgos para aumentarlo, pero una vez que ya tiene cierto capital prefiere no seguir arriesgándolo, ya que las perdidas ahora podrían ser muy grandes por lo que aumenta lo invertido a bajo riesgo. Algo a recalcar es que la gráfica no es perfectamente simétrica con respecto a la diagonal por el efecto de las comisiones de pasar de un tipo de activo al otro. Por el lado de la fracción de consumo esta se mantiene constante en 10% para casi cualquier nivel de capital. Esto es resultado de cómo se planteó el modelo al utilizar una función de utilidad con Aversión Relativa al Riesgo Constante (CRRA)



## Iteración de Política

Para la implementación y el cálculo de la ecuación de Bellman, tuvimos que simular de antemano distintos valores de  $\epsilon \sim N(0,1)$  y luego computar la función valor de la misma forma que para iteración valor. Es decir, computamos el valor promedio de la función valor para un  $\beta$  y  $\delta$  dados sobre todas las simulaciones de los  $\epsilon$  que son  $N_{obs}$ .

Iteración política lo inicializamos con una política inicial para todos los estados, por ejemplo  $\beta = 0.5$  y  $\delta = 0.1$  y manteniendo fija esta política se aplica iteración valor hasta lograr convergencia. Una vez convergido, se actualizan las políticas de la siguiente forma: se exploran todos los distintos  $\beta$  y  $\delta$  y se computa la función valor para cada uno y se chequea si son mejores que los obtenidos con la política inicial. Si para algún  $W_t^f$  y  $W_t^r$  existe un  $\beta$  y  $\delta$  mejor, entonces se actualiza la política original y se procede de vuelta con iteración valor nuevamente hasta lograr convergencia. En caso de que la política no cambie, entonces esta es la política optima y se devuelve los  $\beta^*$  y  $\delta^*$  para cada  $W_t^f$  y  $W_t^r$  con el  $V^*(W_{t+}^r, W_{t+}^f)$ . Además, elegimos Numba, una librería de Python, para implementar la iteración de política porque permite acelerar drásticamente el procesamiento numérico, compilando a código máquina funciones que de otro modo serían lentas en Python puro. Así, podemos experimentar con grillas (grids) más finas y múltiples simulaciones en tiempos razonables, sin alterar los resultados del algoritmo. ✓

Para analizar la sensibilidad del modelo, seleccionamos ocho valores representativos de los hiperparámetros clave: cuatro para  $\alpha$  (0.2, 0.5, 0.9 y 0.997) y cuatro para  $\gamma$  (0.2, 0.5, 0.9 y 1). En cada experimento se varía un solo parámetro a la vez, partiendo del caso base presentado previamente ( $\alpha=0.997$ ,  $\gamma=0.5$ ), para aislar el impacto de cada uno. Así, exploramos desde escenarios de máxima impaciencia ( $\alpha$  bajo) hasta horizonte largo ( $\alpha$  alto), y desde aversión extrema al riesgo ( $\gamma$  bajo) hasta fuerte preferencia por el consumo inmediato ( $\gamma$  alto). Esto permite evaluar cómo cada parámetro, por separado, modifica la política óptima y los resultados de simulación. Bien!

Los resultados obtenidos se visualizan en los siguientes gráficos:

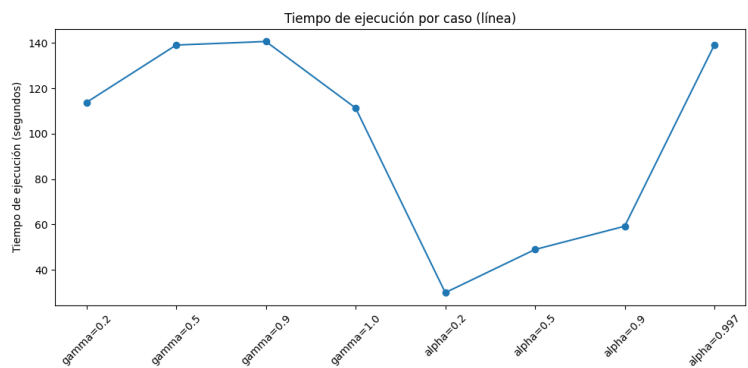


Ilustración 2: Tiempo de ejecución para cada caso variando alpha o gamma

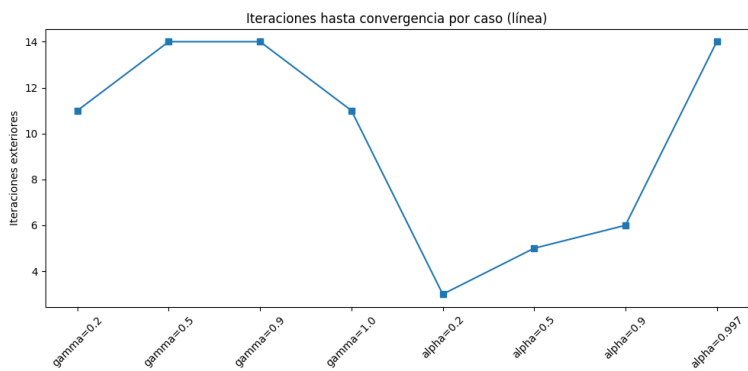


Ilustración 3: N° de iteraciones hasta convergencia por caso

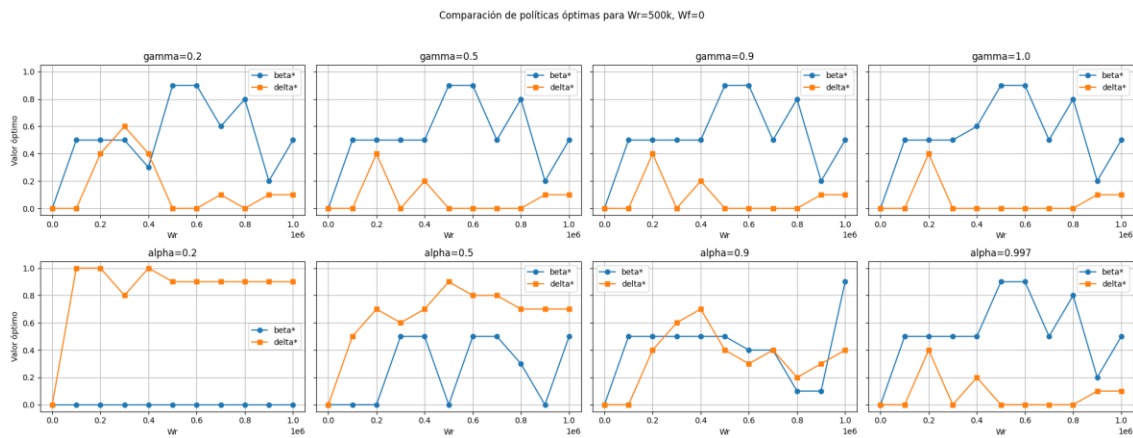


Ilustración 4: Comparación de políticas óptimas con Wf fijo en 0 y Wr 500,000

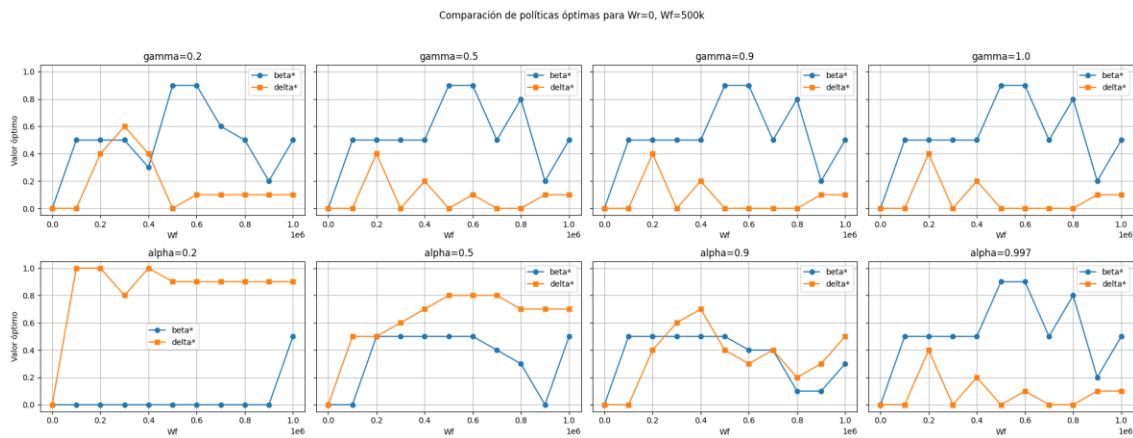


Ilustración 5: Comparación de políticas óptimas con Wr fijo en 0 y Wf = 500,000



## Análisis de tiempo e iteraciones

En el primer y segundo gráfico, se observa cómo varían tanto el tiempo total de ejecución (en segundos) como la cantidad de iteraciones necesarias para que el algoritmo encuentre la política óptima, según los valores elegidos para gamma y alpha.

- Con respecto a  $\gamma$  (primera mitad del eje x): Se ve que para valores bajos de gamma (0.2 y 0.5), tanto el tiempo de cómputo como el número de iteraciones aumentan, alcanzando un máximo alrededor de gamma = 0.9. Con gamma = 1, ambos indicadores bajan levemente, aunque se mantienen relativamente altos. Esto ocurre porque **un gamma bajo implica que el agente le da mucho peso al valor futuro, por lo que el algoritmo necesita más iteraciones y más tiempo para capturar adecuadamente esa utilidad a largo plazo**. En contraste, cuando gamma se acerca a 1 (preferencia por el consumo inmediato), la política converge más rápido porque el valor presente domina, simplificando el proceso de ajuste en cada iteración. **Cómo la aversión al riesgo implica más peso al futuro?**
- Con respecto a  $\alpha$  (segunda mitad del eje x): Un alpha bajo (por ejemplo, 0.2) se traduce en menos iteraciones y menor tiempo de cómputo, ya que el agente descuenta fuertemente el futuro y su política tiende a ser más "corta de vista". Esto permite que la función de valor se estabilice rápidamente. Sin embargo, a medida que alpha aumenta, el algoritmo requiere más iteraciones y más tiempo para converger (particularmente para alpha = 0.997, es decir, casi sin descuento), porque la política debe optimizar teniendo en cuenta recompensas a muy largo plazo, lo que complejiza y enlentece la convergencia. Si bien la relación no es exactamente lineal, la tendencia general se mantiene: a mayor alpha, mayor tiempo y número de iteraciones.

## Análisis de los gráficos de políticas óptimas

En los gráficos de simulación correspondientes a los distintos valores de gamma, se observa un comportamiento claramente diferenciado entre el caso de gamma bajo (0.2) y los casos de gamma medio-alto (0.5, 0.9 y 1.0), y esto ocurre de manera idéntica tanto para el escenario en que toda la riqueza inicial está en el activo libre de riesgo ( $W_r = 0$ ,  $W_f = 500k$ ) como para el caso inverso ( $W_r = 500k$ ,  $W_f = 0$ ). Cuando gamma es bajo (0.2), la política óptima resultante es sumamente conservadora: la fracción de consumo ( $\delta^*$ ) es prácticamente nula en todos los niveles de riqueza, y la fracción de inversión en el activo riesgoso ( $\beta^*$ ) es baja y casi constante a lo largo de toda la grilla, lo cual se traduce visualmente en líneas muy planas o con escasa variabilidad en los gráficos.

riesgo

Esto refleja que el agente, al ser extremadamente adverso al **consumo**, prefiere postergar sistemáticamente el gasto y minimizar la exposición al riesgo. Sin embargo, para gamma = 0.5, 0.9 y 1.0, las políticas óptimas cambian radicalmente y muestran un patrón casi idéntico entre sí: tanto la fracción de inversión en el activo riesgoso como la fracción destinada al consumo presentan variaciones similares ante los cambios de riqueza, y los gráficos exhiben escalones o saltos que se mantienen para los tres valores, independientemente de cómo esté distribuida la riqueza inicial.

Este fenómeno responde a que, a partir de un cierto valor de gamma, la preferencia relativa entre consumo presente y futuro se estabiliza y el perfil de riesgo del agente deja de influir significativamente en la forma de la política óptima. Por tanto, la sensibilidad del modelo a gamma es fuerte sólo cuando gamma es muy bajo, y se vuelve prácticamente nula a partir de valores intermedios, garantizando la robustez de las recomendaciones de política en un rango amplio de aversión al riesgo.

Frente a distintos valores del factor de descuento alpha: Al analizar el comportamiento de las fracciones óptimas de inversión en el activo riesgoso ( $\beta^*$ ) y de consumo ( $\delta^*$ ), se observan patrones claros y diferenciados en ambos escenarios de riqueza inicial ( $W_r = 0$  y  $W_f$  variable, o bien  $W_f = 0$  y  $W_r$  variable). Cuando alpha es bajo, por ejemplo 0.2, el agente actúa de manera sumamente impaciente: la política óptima resulta en una  $\delta^*$  que salta rápidamente de cero a uno apenas la riqueza lo permite, lo que indica que el agente consume prácticamente toda su riqueza tan pronto como puede. En este mismo caso,  $\beta^*$  permanece en cero durante casi todo el rango, incrementándose abruptamente sólo cuando la riqueza es muy alta; es decir, la aversión al riesgo es máxima y la toma de riesgo ocurre solamente en situaciones excepcionales. Al



aumentar alpha a 0.5, se mantiene la tendencia, aunque el salto en delta\* ocurre algo más tarde y beta\* comienza a elevarse antes, pero sigue siendo poco proclive al riesgo y al ahorro.

A medida que alpha se incrementa a valores intermedios-altos como 0.9, la transición en ambos parámetros se suaviza. Delta\* muestra zonas donde el consumo óptimo ya no es exclusivamente todo o nada, sino que adopta valores intermedios en amplios rangos de riqueza, mientras que beta\* pasa de ser casi nulo a elevarse gradualmente conforme aumenta la riqueza, reflejando una mayor disposición a invertir en el activo riesgoso conforme mejora la situación patrimonial del agente. Finalmente, para valores cercanos a uno, como alpha = 0.997, la política óptima se vuelve marcadamente más prudente y paciente: delta\* permanece baja durante casi todo el rango de riqueza y sólo crece suavemente cuando la riqueza es elevada, lo que implica que el agente elige consumir muy poco en el presente para preservar capital hacia el futuro. Simultáneamente, beta\* crece de manera más continua y progresiva, reflejando una toma de riesgo cada vez mayor a medida que se acumula riqueza, pero sin saltos bruscos.

#### Buena análisis de sensibilidad!

Estas tendencias se mantienen tanto si la riqueza inicial está completamente en  $W^r$  como si está en  $W^f$ , ya que la lógica de la política óptima responde más al nivel total de recursos y a la paciencia intertemporal del agente que a la composición inicial de la riqueza. En síntesis, cuanto menor es alpha, más miope y arriesgado es el consumo presente, y cuanto mayor es alpha, más sofisticada, gradual y prudente se vuelve la política, distribuyendo el consumo en el tiempo y diversificando el riesgo de manera más eficiente.

## Q-Learning

El algoritmo comienza inicializando a cero una Q-table que mapea pares de riqueza (riesgosa, libre de riesgo) y acciones (beta, delta) a un valor. Para cada episodio de entrenamiento, parte de un estado de riqueza aleatorio y, en cada paso, selecciona un par de acción (beta, delta) usando una política *epsilon-greedy*: con probabilidad épsilon elige una acción al azar, y si no, selecciona la acción que mayor valor tiene en la función Q para el estado actual. A continuación, el código ejecuta una simulación  $\epsilon \sim N(0,1)$  de esta acción y computa la función valor aproximada para esta simulación, es decir, calcula la recompensa inmediata y el Q-valor máximo posible desde el siguiente estado y utiliza estos valores junto con el factor de descuento  $\alpha$  y la tasa de aprendizaje  $\gamma$  para calcular el nuevo Q-valor, que sobrescribe al anterior en la tabla, completando un ciclo de aprendizaje:

$$\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + \gamma_t \left[ r_t + \alpha \max_{a_{t+1} \in A} \hat{Q}(s_{t+1}, a_{t+1}) - \hat{Q}(s_t, a_t) \right]$$

Con Estado  $s_t: (W_t^r, W_t^f)$ ; Acción  $a_t: (\beta_t, \delta_t)$ ; Recompensa  $r_t: v(\delta_t(W_t^r + W_t^f))$ ;

Tasa de Aprendizaje  $\gamma_t$ ; Factor de Descuento  $\alpha$  y Siguiendo Estado  $s_{t+1}: (W_{t+1}^r, W_{t+1}^f)$ . Por lo tanto, este algoritmo cuenta con muchos hiperparámetros para probar:

- Tasa de Aprendizaje  $\gamma_t$
- Cantidad de episodios
- Cantidad de pasos por episodio
- *Epsilon* (tasa de aceptación de una acción aleatoria)

Para el *learning rate* y el *epsilon* arrancamos con valores altos para favorecer la exploración del algoritmo de distintas estrategias y “zonas”, y luego a lo largo de los episodios vamos bajando la tasa de aprendizaje y el *epsilon* para favorecer más la explotación de buenas soluciones. La cantidad de episodios y pasos por episodio la definimos de antemano y experimentamos.

A continuación, graficamos los resultados obtenidos:

Políticas Óptimas de Inversión y Consumo

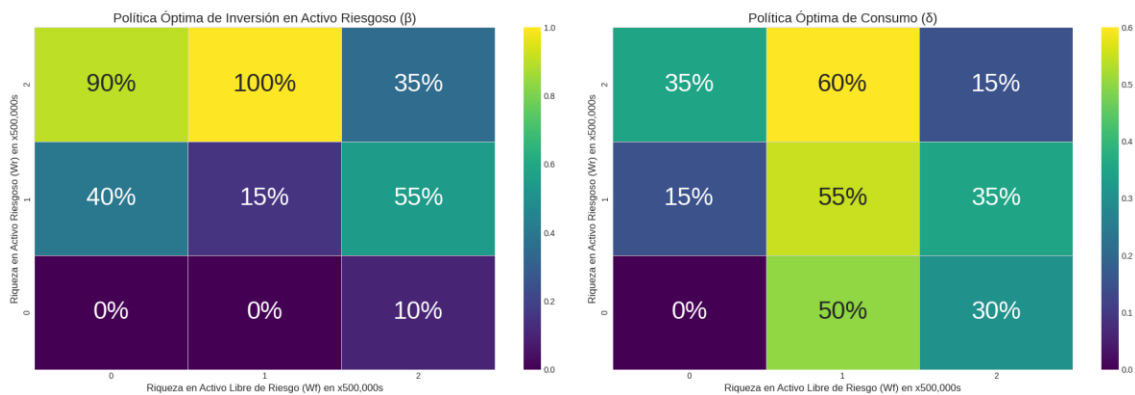


Ilustración 6: Política aproximada obtenida con 3 discretizaciones para  $W^r$  y  $W^f$ .

En la ilustración 6, se observa una política “óptima” (aproximada) obtenida con el algoritmo de Q-Learning para el beta en el grafico izquierdo y el delta en el grafico derecho. En este caso para facilitar la interpretación se hicieron solo 3 discretizaciones para cada  $W_t^f$  y  $W_t^r$  (solo toma los valores 0, 500.000 y 1.000.000) y los betas y deltas se discretizaron 21 veces cada uno. El algoritmo lo corrimos con la mejor configuración de hiperparámetros que hayamos (lo describimos más adelante). Por ejemplo, cuando se tiene 1 M de ambos activos, se destina un 15 % del dinero total (2 M) para el consumo y el resto para inversión. De lo que queda para invertir se destina el 35% para el activo riesgoso (descontando por el costo de transacción) y el resto al activo libre de riesgo. Cuando se tiene 500 K de  $W_t^f$  y 1 M de  $W_t^r$ , se destina el 60% para consumo y el resto se destina completamente al activo riesgoso descontando por los costos de transacción. Otro ejemplo, cuando se tiene solo 500 K en  $W_t^f$  y nada en el activo riesgoso, entonces se consume el 50 % de esta riqueza (250 K) y el resto se destina completamente al activo libre de riesgo (porque beta es cero). De esta forma, pudimos obtener una serie de políticas aproximadas de una manera rápida y eficiente.

Para comparar los resultados con los de un algoritmo exacto, comparamos el mismo ejemplo con los resultados obtenidos al correr iteración valor:

Políticas Óptimas de Inversión y Consumo (Iteración de Valor)

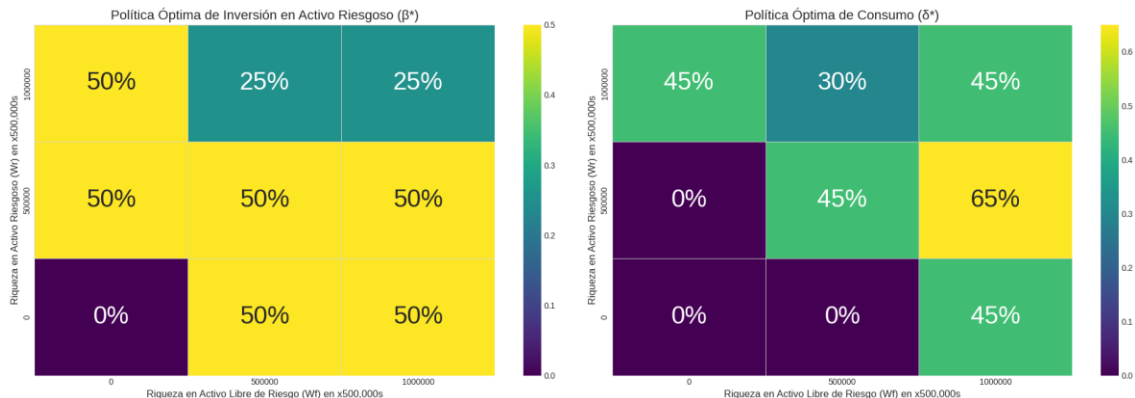


Ilustración 7: : Política óptima obtenida con 3 discretizaciones para  $W^r$  y  $W^f$ .

Se observa que las políticas optimas difieren bastante para algunos valores de  $W_t^r$  y  $W_t^f$  del caso de Q-Learning. Por ejemplo, si tenemos 500 K de  $W_t^r$  y  $W_t^f$ , iteración valor nos dice que debemos destinar el 45 % de esta riqueza para consumo (comparado con Q-Learning que era 55%) y del resto, el 50 % destinarlo al activo riesgoso mientras que en Q-Learnig el 15 % se destina al activo riesgoso. Esto muestra claramente que los resultados de Q-Learning son aproximados pero rápidos comparados a los de iteración valor que son exactos pero lentos en obtener.

Cuanto se pierde en ganancia en cambio de un algoritmo más rápido?

También hicimos el mismo análisis con la misma configuración del algoritmo de Q-Learning, pero discretizando aún mas  $W_t^r$  y  $W_t^f$ , en esta caso en 11 particiones y obtuvimos el siguiente grafico:

Políticas Óptimas de Inversión y Consumo



Ilustración 8: Política óptima obtenida con 11 discretizaciones para  $W_r$  y  $W_f$ .

Ahora la interpretabilidad es mucho más complicada ya que depende de los distintos valores que tome la riqueza de cada activo. Sin embargo, esto lo podemos discretizar aún más lo que permite obtener políticas más precisas, pero a un costo computacional mayor.

También, para el caso con 11 discretización, pusimos a prueba al algoritmo de Q-Learning para evaluar que tan bien aproxima en general a las funciones valores obtenidas con iteración valor para el mismo problema. Esto significa, que se evalúa las funciones valores obtenidas sobre todos los  $W_s$  y se computa el error cuadrático medio (MSE) como métrica de función de perdida para medir que tan bien aproxima. Los resultados se resumen en el siguiente grafico:

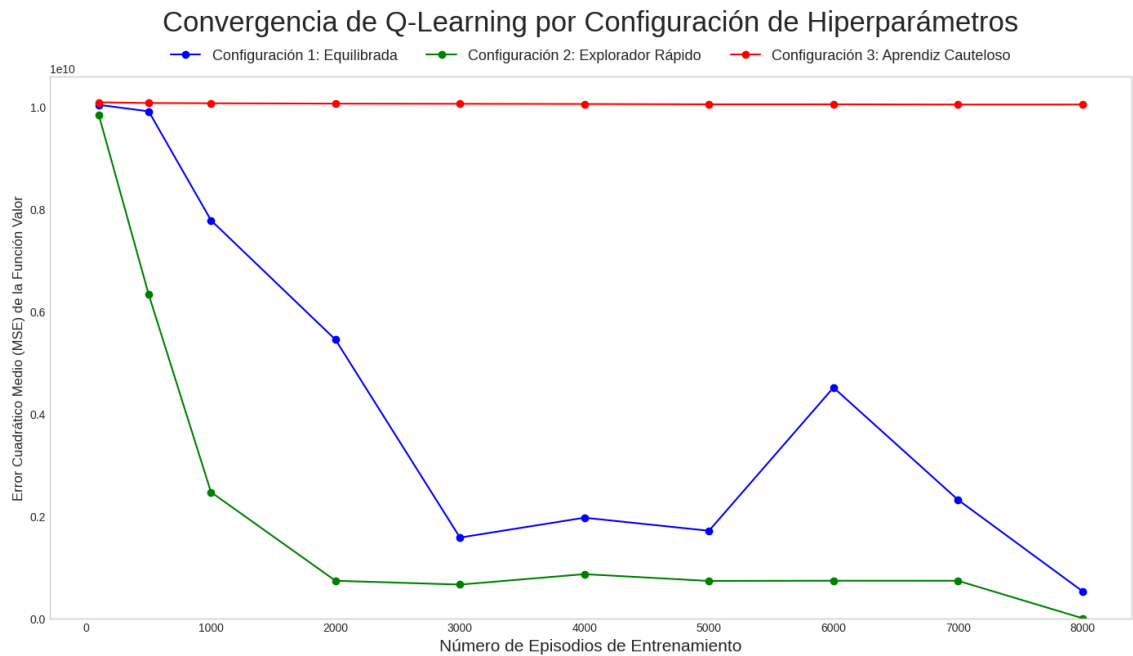


Ilustración 9: Convergencia del algoritmo Q-Learning a la función Valor de Iteración Valor al aumentar el número de episodios con distintas configuraciones de hiperparámetros.

Para testear al algoritmo armamos tres configuración distintas de hiperparámetros que se resumen en la siguiente tabla:

```
configuraciones = {
  "Configuración 1: Equilibrada": {
    "lr_inicial": 0.5, "lr_decay": 0.9995,
    "epsilon_decay": 0.999
  },
  "Configuración 2: Explorador Rápido": {
    "lr_inicial": 0.99, "lr_decay": 0.9999, # Aprende agresivamente por más tiempo
    "epsilon_decay": 0.998 # Deja de explorar mucho más rápido
  },
  "Configuración 3: Aprendiz Cauteloso": {
    "lr_inicial": 0.2, "lr_decay": 0.9995, # Aprende de forma conservadora
    "epsilon_decay": 0.99999 # Explora durante muchos más episodios
  }
}
```

Buen experimento!

La configuración 1 (línea azul del gráfico), llamada “Equilibrada”, tiene un *learning rate* inicial intermedio y decae suavemente junto con el *epsilon decay*. La idea es que esta sea de benchmark para comparar otros dos configuraciones más agresivas. La segunda configuración, denominada “Explorador rápido” (línea verde del gráfico), tiene un *learning rate* inicial mucho más alto y decae más lentamente comparado a la configuración equilibrada. Esto permite aprender mucho más rápido a lo largo del tiempo, también el *epsilon decay* se redujo ligeramente para favorecer una exploración más corta y concentrarse más en políticas *greedy*. Por último, la configuración 3, llamada “Aprendiz cauteloso” (línea roja del gráfico), tiene un *learning rate* inicial mucho más bajo comparado con la configuración de equilibrio. Esto permite que aprenda de forma más conservadora y muy de a poco. El *learning rate decay* es igual al de la configuración de equilibrio, pero el *epsilon decay* es más alto para favorecer una mayor exploración. Todas estas configuración se evaluaron a lo largo de distinta cantidades de episodios y cada episodio hace 1000 pasos en cada pasada para todas las configuraciones.

Como resultado, la configuración de “Aprendiz cauteloso” termino siendo la peor ya que no aprendió casi nada a lo largo de los episodios, esto se debe a que la tasa de aprendizaje era muy baja y por más que explore mucho que las otras configuraciones, esto no es suficiente para poder aprender de los datos. Por otro lado, la configuración “Explorador Rápido” es la mejor de todas ya que al tener un *learning rate* inicial alto y que decae lentamente le permite aprovechar cada vez más la cantidad de episodios para aprender mucho mejor de los datos y el *epsilon decay* bajo le permite dejar de explorar rápidamente áreas de poco interés para concentrarse en zonas de exploración más prometedoras. Esta configuración es ideal y es la que usamos para todas las ejecuciones de este algoritmo. Por último, la configuración “Equilibrada” tiene un comportamiento intermedio y muy inestable. Aunque el error general tiende a disminuir, lo hace de forma mucho más lenta y errática que en la configuración 2. Se observa una caída inicial, seguida de un estancamiento e incluso un pico de error inesperado alrededor de los 6000 episodios, antes de volver a bajar. Esta volatilidad sugiere que el proceso de aprendizaje no es estable; la combinación de exploración y actualización no es tan eficiente y, en ocasiones, puede llevar al algoritmo a "desaprender" temporalmente o a alejarse de la solución óptima debido a la aleatoriedad de las simulaciones. Si bien aprende, es claramente inferior a la configuración del "Explorador Rápido".

En conclusión, el gráfico demuestra de manera concluyente la importancia crítica del ajuste de hiperparámetros. La configuración "Explorador Rápido" es la clara vencedora, convergiendo rápidamente a una solución de bajo error. La configuración "Aprendiz Cauteloso" ilustra el peligro de un aprendizaje demasiado lento, mientras que la "Equilibrada" muestra que incluso una configuración aparentemente razonable puede ser ineficiente e inestable.

## Análisis y futuras direcciones

Las direcciones futuras de este trabajo se centran en expandir el realismo del modelo y superar las limitaciones computacionales encontradas. Una extensión natural sería incorporar múltiples activos de riesgo (acciones, bonos, criptomonedas) y parámetros de mercado estocásticos como las tasas de interés o la volatilidad, lo que reflejaría mejor la dinámica financiera real. Sin embargo, esto agrava la "maldición de la dimensionalidad" que ya identificamos, haciendo que el enfoque de Q-table sea impracticable. Por lo tanto, el principal desafío y la dirección más prometedora es la transición de métodos tabulares a la aproximación de funciones, específicamente implementando un algoritmo de Deep Q-Learning (DQN). Al usar una red neuronal para estimar la función Q, se podría manejar un espacio de estados infinitamente más grande y continuo, aunque esto introduciría nuevos desafíos relacionados con el diseño de la arquitectura de la red, la estabilidad del entrenamiento y la mayor complejidad en el ajuste de hiperparámetros.